

Final Evaluation - *RedGobblingTurkey*

Peng Liu (*peng.liu2*)

Valentina Zingarello (*valentinazingarello*)

December 8, 2025

1 Introduction

RedGobblingTurkey is a movie-question answering agent capable of handling factual queries, performing lightweight embedding-based inference, providing structured recommendation responses, and retrieving images of movies and actors from the provided multimedia dataset. The final version of the system implement also the image-handling pipeline: the agent can detect whether the user is requesting a movie poster or an actor portrait, resolve entity names, select the most appropriate image among several candidates, and return it inline via the Speakeasy platform.

The remaining components (SPARQL factual lookup, TransE-based inference, and heuristic recommendation engine) complement the image pipeline and aim to provide a robust, user-friendly agent.

2 Capabilities

Overview Diagram

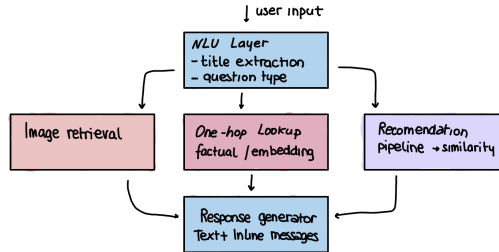


Figure 1: System architecture highlighting the image-handling pipeline.

- **Multimedia Questions**

The agent can return actor portraits and movie posters from the dataset using `image_handler.py`. It detects when the user is requesting an image

(e.g., “*Show me the poster of Forrest Gump*”, “*What does Denzel Washington look like?*”), resolves the corresponding entity, retrieves all associated images, applies priority heuristics (poster first, publicity photos for actors), and returns the selected file.

- **Factual Questions**

Using SPARQL over a local RDF graph, the system answers one-hop questions involving properties such as directors, screenwriters, genres, countries, awards, and release dates.

- **Embedding Questions**

When explicitly instructed to “use embeddings,” the system performs TransE-style vector arithmetic to produce an approximate answer for the requested relation.

- **Recommendation Questions**

The agent provides structured recommendation responses using a multi-stage heuristic pipeline based on shared attributes (directors, genres, countries), dominance patterns, and explicit user constraints.

3 Adopted Methods

- **Image Handling Pipeline**

Implemented in `image_handler.py`, this module loads the multimedia dataset, builds efficient mappings between IMDb identifiers and Wikidata QIDs, aggregates images by entity, parses user queries to detect image intent, performs fuzzy resolution of entity names, and selects an image using domain-specific heuristics. Importantly, it performs *no* OCR or visual analysis: image retrieval is entirely metadata-driven.

- **RDFLib** [3]

Used for executing SPARQL queries on the local RDF dump to answer factual questions.

- **TransE-style Embeddings** [1]

Pre-trained entity and relation vectors support approximate inference when factual data is missing, using nearest-neighbor search on the predicted embedding.

- **NumPy** [2]

Used throughout the embedding pipeline for vector manipulation and storage.

- **scikit-learn Pairwise Distances** [4]

Provides efficient nearest-neighbor lookup for the embedding-based reasoning component.

- **Speakeasy SDK**
Enables real-time message handling, inline image delivery, and multi-room communication.

4 Examples

- **Multimedia Question**
For the query “*Show me a picture of Halle Berry*”, the agent detects image intent, maps the name to a QID, retrieves all associated portraits, prioritizes solo-cast publicity images, and returns the best candidate.
- **Factual Question**
For “*Who is the director of Good Will Hunting?*”, the system resolves the title, queries property P57, and returns *Gus Van Sant*.
- **Embedding Question**
For “*Use embeddings: who is the director of Apocalypse Now?*”, the TransE reasoning module computes $e + r$ and returns the nearest director candidate.
- **Recommendation Question**
Given “*Recommend movies similar to La Dolce Vita*”, the agent identifies shared attributes (e.g., Fellini’s direction, Italian cinema, genre), selects the strongest pattern, and presents recommendations accordingly.

5 Additional Features

Beyond the core functionalities, the system also supports:

- Robust normalization of punctuation and smart quotes.
- Fuzzy entity matching for noisy user input.
- Multi-level fallback strategies (factual $\rightarrow$ embeddings $\rightarrow$ recommendation).
- Human-like clarification suggestions when entity resolution fails.
- Pre-indexing of the multimedia dataset to ensure efficient image lookup at runtime.

6 Conclusions

This project delivers a fully working movie-question agent capable of handling factual, embedding-based, recommendation and multimedia queries. The focus of this version is the image-handling system, which integrates metadata-driven entity resolution and structured image retrieval. Through the development of

RedGobblingTurkey, we gained practical experience in combining knowledge-graph querying, embedding-based inference, recommendation strategies and large-scale multimedia indexing into a unified and user-facing system. This taught us how to transform modular technical tools into a end-to-end solution capable of handling real-world ambiguity and noisy user input.

References

- [1] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. In C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013.
- [2] <https://numpy.org/>.
- [3] <https://rdflib.readthedocs.io/>.
- [4] <https://scikit-learn.org/stable/>.